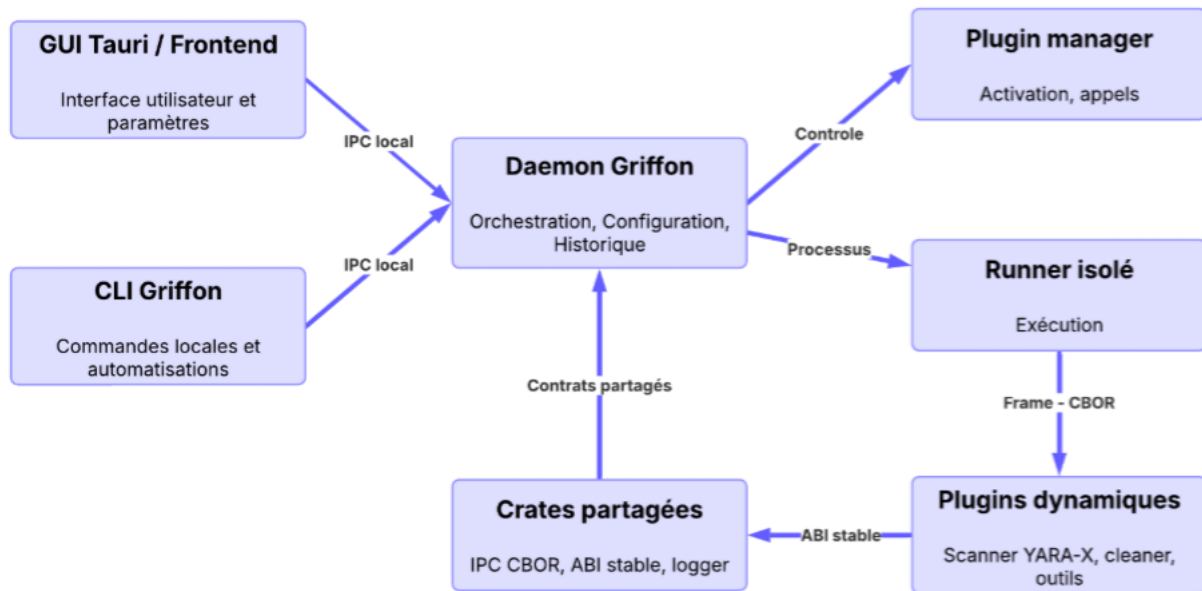


KPI 1 DOSSIER

Evaluating and Integrating New Technologies

Griffon Project — Modular Security Platform for Linux



Overview of the architecture implemented in Griffon

Condensed version — July 2026

Technology watch • Comparison • POC • Integration • Measurement • Sharing

1. Executive Summary

Objective: demonstrate that Griffon identified real technical needs, compared several solutions, experimented, integrated the selected choices, and measured their impact.

Griffon evolved from a monolithic prototype into a platform composed of a daemon, a Tauri GUI, a CLI and dynamic plugins. Technology watch directly guided five structural choices: stable ABI, Frame + CBOR IPC, YARA-X, Tauri, and Grafana/SQLite.

Need	Selected choice	Concrete evidence
Load independent Rust plugins	abi_stable	Plugin Manager, shared interface, .so plugins
Make local communication reliable	12-byte frame + CBOR	Hello/Call/Result/Error, request_id, 1 MB limit
Integrate a detection engine	YARA-X	Rust scanner, rules, archives, quarantine
Provide a desktop application	Tauri	GUI connected to the daemon and plugins
Measure performance	JSON → SQLite → Grafana	Cleaner v0.0.9 / v0.1.0 comparison

Key Results

- Two structural POCs adopted: Plugin Manager/stable ABI and IPC protocol v1.
- YARA-X integrated instead of a direct YARA C integration.
- Cleaner v0.1.0: +8.3% across two comparable safe-mode runs.
- The 10,000-path incident was fixed through sorting, aggregation, and payload limits.
- Regular technology watch through RSS/Discord and direct outreach to the Rust community on Reddit.

Conclusion: the KPI is covered by evidence of technology watch, experimentation, integration, measurement, and sharing.

2. Context and Technology Watch Method

2.1 Core Question

How can we build a Linux platform extensible through Rust plugins while maintaining a stable contract, bounded communications, effective detection, and a reproducible measurement process?

2.2 Monitored Scope

Area	Monitored topics
Rust / plugins	ABI, FFI, dynamic libraries, abi_stable, WebAssembly
Architecture	daemon, runner, isolation, contract versioning
IPC	framing, CBOR, correlation, size limits
Security	YARA-X, Rust dependencies, permissions, rules
Desktop	Tauri, frontend/backend, trust model
Performance	metrics, SQLite, Grafana, regressions
Deployment	systemd, .deb/.rpm, group permissions

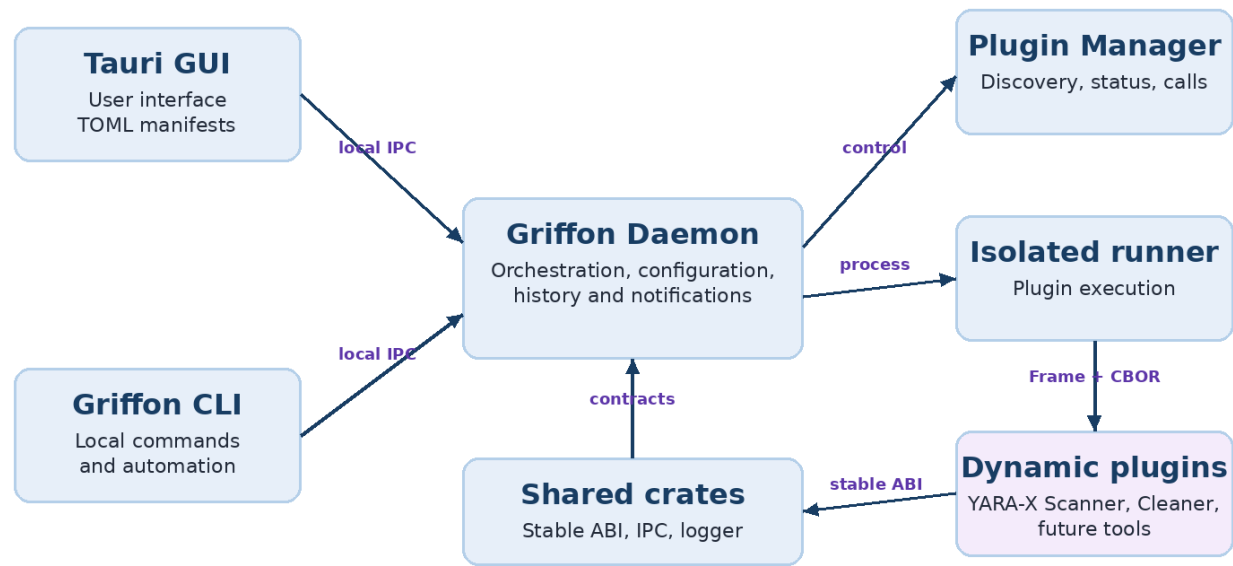
2.3 Applied Process

Step	Action
1	Define a project-related problem.
2	Review official sources and repositories.
3	Compare alternatives.
4	Run a time-boxed POC.
5	Measure and document.
6	Adopt, reject, or defer.
7	Integrate through an issue/PR and update the documentation.

2.4 Sources

Official Rust documentation, RFC 8949, abi_stable, YARA-X, Tauri and Grafana; the Griffon GitHub repository; JSON reports and the Grafana dashboard; Rust Foundation, The Hacker News and This Week in Rust feeds; and Reddit discussions.

3. Architecture Evolution



Central daemon, isolated runner and dynamic plugins

Verifiable Evolution

Date	Evidence	Result
Nov. 2025	PR #90	First working version of the Plugin Manager.
Dec. 2025	PR #91	IPC protocol v1.
Mar. 2026	PR #118	New daemon architecture, still experimental.
Apr. 2026	PR #122	Working basic daemon.
May–Jun. 2026	PR #149	Packaging and CLI/GUI execution without sudo.
Jun. 2026	PR #175 / #180	History and notifications integrated from daemon to GUI.

Main progression: monolithic POC → modular architecture → packaging → instrumentation → measurement.

4. Technology Decisions

4.1 Plugins: `abi_stable`

Option	Advantage	Limitation	Decision
Native Rust ABI	Simple	No stability guarantee	Rejected
C ABI	Stable and multi-language	Manual conversions and ownership handling	Alternative
<code>abi_stable</code>	ABI-safe types and load-time checks	Specific types	Adopted
WebAssembly	Isolation and portability	Significant runtime and migration cost	Deferred

The contract is intentionally minimal: `init()` returns metadata, while `handle_message()` receives a command and returns a response. Standard Rust types do not cross the ABI boundary.

4.2 IPC: Frame + CBOR

Element	Value / role
Header	12 bytes: magic, version, type, request_id, payload_len
Messages	Hello, HelloOk, Call, Result, Heartbeat, Error
Serialization	CBOR via <code>serde_cbor</code>
Security	Validation of magic, version, type, and <code>MAX_PAYLOAD = 1 MB</code>

CBOR is used for the Plugin Manager <-> runner channel. JSON remains useful for manifests and some human-readable responses. A compact format does not replace application-level pagination.

4.3 Detection: YARA-X

Criterion	YARA C	YARA-X
Rust integration	FFI and native build	Direct Rust API
Memory safety	Depends on C code/wrapper	Core written in Rust
Maturity	Highly established	Newer, versions must be monitored
Griffon choice	Not selected directly	Adopted with pinned versions and CI

4. Technology Decisions — Continued

4.4 Desktop Application: Tauri

Option	Main strength	Risk / cost	Decision
Tauri	Rust backend + web frontend	Permissions and WebKitGTK dependencies	Adopted
Electron	Complete web ecosystem	Runtime less aligned with Rust	Not selected
GTK / Rust toolkit	Native interface	Longer development time	Not selected

The TOML manifest describes identity, state, components, and interactions. Griffon can therefore build a plugin interface without a dedicated React page. The next priority is to make these TOML files easier for a new contributor to write and validate.

4.5 Measurement: JSON, SQLite and Grafana

Each Cleaner execution produces a versioned report. The data is imported into SQLite and compared in Grafana. Grafana is used for reporting, not real-time monitoring.

Metric	Use
files_scanned_total	Workload traversed
files_cleaned_total	Actual action performed
run_duration_seconds	Total time
bytes_freed_per_second	Comparative throughput
errors_by_type	Robustness and permissions

4.6 Decision Register

Choice	Status	Reassessment
abi_stable	Adopted	If sandboxing or multi-language support is needed
Frame + CBOR	Adopted	When protocol v2 is designed
YARA-X	Adopted	At each major version
Tauri	Adopted	If WebView or packaging constraints arise
Grafana + SQLite	Adopted	If CI automation or data volume increases
WebAssembly	Deferred	After stabilizing the plugin SDK

5. POCs and Concrete Integrations

Experiment	Question	Result	Decision
Plugin Manager + ABI	Can a Rust .so be loaded independently?	init(), function discovery, and operational calls	Adopted
Frame + CBOR	Can typed and bounded messages be transported?	12-byte header, request_id, handshake, 1 MB limit	Adopted

YARA-X migration	Can direct C integration be avoided?	Rust scanner integrated; versioning difficulty identified	Adopted with version pinning
------------------	--------------------------------------	---	------------------------------

Integration Evidence

- Real plugins and a plugin template are available in the repository.
- TOML manifests are used to build the state store and GUI.
- The GUI and CLI are connected to the daemon.
- Activation, notifications, and history are managed by the daemon.
- .deb/.rpm packaging and a systemd service.
- Dedicated guides covering plugin architecture, the Rust backend, interactions, packaging, and IPC.

Validated Criteria

Criterion	Status
Plugin can be compiled separately from the daemon	Validated
Function call through the host	Validated
Controlled protocol error	Validated
Documented installation	Validated
Measurement across versions	Validated
Sharing through documentation and GitHub	Validated

6. Cleaner Plugin Benchmark

6.1 Exact Measurements Provided

Date	Version	Mode	Files	Volume	Duration	Throughput
03/07	v0.0.9	aggressive	107 563	15.16 GB	1.460 s	10.385 GB/s
03/07	v0.0.9	safe	107 563	13.50 GB	1.279 s	10.557 GB/s
04/07	v0.0.9	aggressive	108 793	15.63 GB	1.489 s	10.500 GB/s
04/07	v0.0.9	safe	108 793	13.52 GB	1.163 s	11.628 GB/s
05/07	v0.1.0	safe	110 388	14.29 GB	1.135 s	12.591 GB/s

Exact comparison: 11.628 GB/s in v0.0.9 safe versus 12.591 GB/s in v0.1.0 safe, a +8.28% improvement.



Grafana dashboard — throughput by version and mode

The aggregated dashboard suggests approximately +29%, but this remains a visual estimate until the SQL query or aggregated export is attached.

6.2 Interpretation

- Throughput increases despite a slightly higher number of files in v0.1.0.

- Aggressive mode cleans thousands of files and encounters more permission errors.
- files_cleaned_total = 0 in safe mode must be documented as dry-run behavior or no deletion.

7. Lessons Learned: Response Containing More Than 10,000 Paths

Incident: the Cleaner returned a massive list of paths as JSON. The daemon could crash while transferring this response to the frontend.

Layer	Cause	Effect
Application logic	All paths were returned without a limit	Linear growth of the result
Serialization	Repeated JSON keys and path prefixes	Unnecessarily large payload
Transport	No pagination or aggregation	A single message contained everything
Frontend	List difficult to render	Higher memory cost and degraded UX

Fix

- Select the largest files.
- Aggregate by root path or category.
- Keep summary metrics.
- Enforce a maximum IPC payload size.
- Separate the summary view from targeted deletion.

Lesson: switching from JSON to CBOR is not enough when the data model remains unbounded. Reducing the information before serialization was the main optimization.

Impact

Before	After
More than 10,000 paths in one response	Sorted and limited results
Risk of daemon crash	Controlled error or payload
Heavy frontend rendering	Actionable summary
No application-level volume rule	Aggregation + MAX_PAYLOAD

8. Documentation, Technology Watch and Community

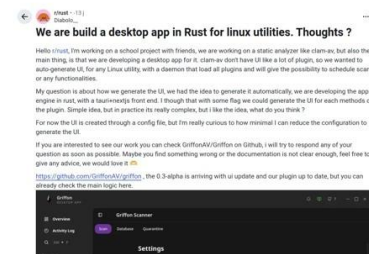
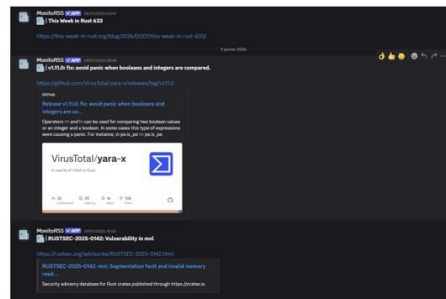
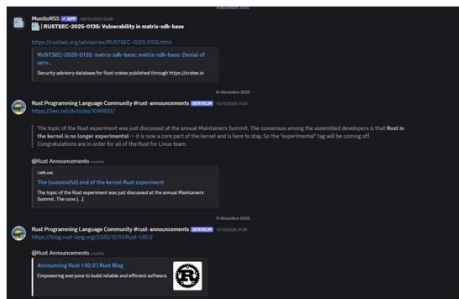
8.1 Knowledge Capture

Document	Contribution
README / Installation	Build, execution, repository structure, and releases
Plugin Architecture / UI / Interactions	.so + .toml model and GUI construction
Rust Backend / Plugin Guide	ABI, init(), handle_message(), and errors
IPC Protocol	Format, handshake, limits, and security
YARA / Grafana	Technology decisions and performance monitoring
POC / PERF templates	Shared experimentation and measurement method

8.2 Technology Watch and Openness

Technology watch and community participation

☐	STATUS	TITLE	URL	ADDED ON	SHARED WITH ME
☐	✔	The Rust Foundation	https://rustfoundation.org/feed/	2020-04-09	Configure >
☐	✔	The Hacker News	https://feeds.feedburner.com/Th...	2020-04-09	Configure >
☐	✔	the week in rust	https://this-week-in-rust.org/rss.xml	2025-10-17	Configure >



RSS feeds, Discord relay, and r/rust publication

- Monitored feeds: Rust Foundation, The Hacker News, and This Week in Rust.
- Discord relay: RUSTSEC advisories and Rust/YARA-X releases.
- r/rust post: technical question about GUI generation and plugin architecture.
- Public repository, Apache-2.0 license, guides, and calls for contribution.

9. Impact and KPI Compliance

9.1 Impacts

Area	Impact
Performance	Versioned metrics, Grafana, +8.3% over two comparable runs
Robustness	Framing, 1 MB limit, error handling, result aggregation
Security	Rust, YARA-X, reduced sudo usage, protocol validation
Maintainability	Shared crates, minimal contract, declarative GUI, guides
Development time	Initial platform cost, followed by reuse for each plugin
Openness	RSS/Discord, Reddit, public repository, and contribution workflow

9.2 Compliance Matrix

Requirement	Evidence	Status
Active technology watch	RSS, Discord, GitHub, official sources	Covered
Comparative analysis	ABI, IPC, YARA-X, GUI	Covered
Benchmark	JSON, SQLite, Grafana	Covered
1–2 POCs	Plugin Manager and IPC v1	Covered
Integration	abi_stable, CBOR, YARA-X, Tauri	Covered
Project updates	Code, docs, packaging, releases	Covered
Community participation	r/rust post and public repository	Covered
Impact summary	Performance, security, maintenance	Covered

The dossier meets the mandatory objectives. Remaining actions mainly strengthen reproducibility and traceability rather than KPI coverage.

10. Limitations and Next Actions

Priority	Action	Benefit
P0	TOML <-> init() <-> handle_message() validator	Detect inconsistencies before installation
P0	Pagination and application-level limits	Prevent unbounded responses
P0	Automatic SQL/CSV benchmark export	Make results auditable
P1	Explicit ABI and protocol versioning	Avoid breaking plugins
P1	Document YARA-X rule licenses and sources	Secure the supply chain
P1	Reproducible tests: machine, cache, repetitions	Strengthen scientific validity
P2	WebAssembly / WASI POC	Evaluate future sandboxing

Conclusion

Griffon demonstrates a complete technology-watch process: a problem is identified, several solutions are compared, a POC is carried out, the selected solution is integrated, and its effects are measured and documented. The choices of `abi_stable`, `Frame + CBOR`, `YARA-X`, `Tauri`, and `Grafana` address concrete project constraints rather than following trends.

The transition from a monolithic prototype to a modular platform is the main proof. Cleaner measurements, the 10,000-path incident, pull requests, guides, RSS monitoring, and the Reddit post show that the approach is applied to code, quality, and knowledge sharing.

Present to the jury: modular architecture, comparison of choices, two POCs, the 10,000-path incident, the Cleaner benchmark, and evidence of sharing.

Main References

Rust Reference — ABI; `abi_stable`; RFC 8949 CBOR; YARA-X documentation; Tauri architecture/security; Grafana dashboards; Griffon documentation and GitHub history.

Main GitHub evidence: PRs #90, #91, #118, #122, #149, #175, and #180; Cleaner issues #80–89 and #134; Scanner issue #130.